📚 DOCUMENTACIÓN COMPLETA - PARTE 2

OpenTalkWisp - Documentos de IA, WhatsApp y Frontend

📄 **docs/04-AI-INTEGRATION.md**
Copiar

```
# 🤖 INTEGRACIÓN DE INTELIGENCIA ARTIFICIAL
## OpenTalkWisp - Sistema de IA Conversacional

---

## 📋 Tabla de Contenidos

1. [Introducción](#introducción)
2. [Arquitectura de IA](#arquitectura-de-ia)
3. [OpenAI Integration](#openai-integration)
4. [LangChain Implementation](#langchain-implementation)
5. [Flujos Conversacionales](#flujos-conversacionales)
6. [Lead Scoring con IA](#lead-scoring-con-ia)
7. [Análisis de Sentimiento](#análisis-de-sentimiento)
8. [RAG (Retrieval Augmented Generation)](#rag)
9. [Prompts del Sistema](#prompts-del-sistema)
10. [Optimización y Costos](#optimización-y-costos)

---

## 🎯 Introducción

El sistema de IA de OpenTalkWisp utiliza modelos de lenguaje avanzados para:

- ✅ Generar respuestas automáticas contextuales
- ✅ Analizar sentimiento de conversaciones
- ✅ Calificar leads automáticamente
- ✅ Crear flujos conversacionales dinámicos
- ✅ Resumir conversaciones largas
- ✅ Sugerir respuestas a agentes humanos
- ✅ Extraer información estructurada de mensajes

### Stack de IA

| Tecnología | Versión | Propósito |
|------------|---------|-----------|
| **OpenAI API** | Latest | Modelo principal (GPT-4) |
| **LangChain** | 0.1.x | Orquestación de LLMs |
| **Pinecone** | Latest | Vector database (RAG) |
| **Tiktoken** | Latest | Conteo de tokens |
| **Zod** | 3.x | Validación de schemas |
```

```
---

## 🤖 Arquitectura de IA

### Diagrama de Componentes
```

┌───┐
FRONTEND │ │ (Usuario/Agente solicita respuesta o análisis) │
└─────────────────────────┬───────────────────────────────────────┘
│ ▼
┌───┐
AI SERVICE LAYER │ │ ┌──────────────────┐ ┌──────────────────┐
┌──────────────┐ │ │ │ Response │ │ Sentiment │ │ Lead │ │ │ │ Generator │ │
Analyzer │ │ Scoring │ │ │ └──────────────────┘ └──────────────────┘
└──────────────┘ │
└─────────────────────────┬───────────────────────────────────────┘
│ ▼
┌───┐
LANGCHAIN ORCHESTRATION │ │ ┌──────────────────┐ ┌──────────────────┐
┌──────────────┐ │ │ │ Chains │ │ Memory │ │ Tools │ │ │ └──────────────────┘
└──────────────┘ └──────────────┘ │
└─────────────────────────┬───────────────────────────────────────┘
│
┌──────────────────┴──────────────────┐ │ │ │ ▼ ▼ ▼ ┌──────────────────┐
┌──────────────┐ ┌──────────────┐ │ OpenAI │ │ Pinecone │ │ Database │ │ API
│ │ (Vectors)│ │ (Context) │ └──────────────┘ └──────────────┘
└──────────────┘

Copiar

```
### Flujo de Procesamiento
```

Mensaje Entrante │ ▼ ┌──────────────────┐ │ Pre-procesamiento│ │ - Limpieza │
│ - Normalización │ └──────────┬───────────┘ │ ▼ ┌──────────────────┐ │
Análisis Inicial│ │ - Idioma │ │ - Intención │ │ - Entidades │ └──────────┬──────────┘
│ ▼ ┌──────────────────┐ │ Búsqueda RAG │ │ (si aplica) │
└──────────┬──────────┘ │ ▼ ┌──────────────────┐ │ Generación LLM │ │ -
Contexto │ │ - Prompt │ │ - Respuesta │ └──────────┬──────────┘ │ ▼
```
```

```
┌──────────────────────────────┐ │ Post-procesamiento │ │ - Validación │ │ - Formateo │
└──────────────┬───────────────┘ │ ▼ Respuesta Final
```

Copiar

```
---

## 🔌 OpenAI Integration

### Configuración Inicial

#### Archivo: `apps/backend/src/modules/ai/config/openai.config.ts`

```typescript
import { Injectable } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import OpenAI from 'openai';

@Injectable()
export class OpenAIConfig {
  private readonly client: OpenAI;

  constructor(private configService: ConfigService) {
    this.client = new OpenAI({
      apiKey: this.configService.get<string>('OPENAI_API_KEY'),
      organization: this.configService.get<string>('OPENAI_ORG_ID'),
      maxRetries: 3,
      timeout: 30000, // 30 segundos
    });
  }

  getClient(): OpenAI {
    return this.client;
  }

  // Configuraciones por tipo de tarea
  getModelConfig(task: 'chat' | 'analysis' | 'embedding') {
    const configs = {
      chat: {
        model: 'gpt-4-turbo-preview',
        temperature: 0.7,
        max_tokens: 500,
        top_p: 1,
        frequency_penalty: 0,
        presence_penalty: 0,
      },
      analysis: {
        model: 'gpt-4-turbo-preview',
        temperature: 0.3, // Más determinista para análisis
        max_tokens: 300,
        top_p: 1,
        frequency_penalty: 0,
        presence_penalty: 0,
      },
      embedding: {
```
```

```
        model: 'text-embedding-3-small',
        dimensions: 1536,
      },
    };

    return configs[task];
  }
}
```

Servicio Base de OpenAI

Archivo: `apps/backend/src/modules/ai/services/openai.service.ts`

Copiar

```
import { Injectable, Logger } from '@nestjs/common';
import { OpenAIConfig } from '../config/openai.config';
import { ChatCompletionMessageParam } from 'openai/resources/chat';
import { encoding_for_model } from 'tiktoken';

interface ChatOptions {
  temperature?: number;
  maxTokens?: number;
  systemPrompt?: string;
}

@Injectable()
export class OpenAIService {
  private readonly logger = new Logger(OpenAIService.name);
  private readonly client;

  constructor(private openaiConfig: OpenAIConfig) {
    this.client = this.openaiConfig.getClient();
  }

  /**
   * Genera una respuesta de chat
   */
  async generateChatResponse(
    messages: ChatCompletionMessageParam[],
    options: ChatOptions = {},
  ): Promise<string> {
    try {
      const config = this.openaiConfig.getModelConfig('chat');

      const completion = await this.client.chat.completions.create({
        model: config.model,
        messages: options.systemPrompt
          ? [{ role: 'system', content: options.systemPrompt }, ...messages]
          : messages,
        temperature: options.temperature ?? config.temperature,
        max_tokens: options.maxTokens ?? config.max_tokens,
        top_p: config.top_p,
        frequency_penalty: config.frequency_penalty,
        presence_penalty: config.presence_penalty,
      });

      const response = completion.choices[0]?.message?.content || '';
```

```
    // Log de uso de tokens
    this.logger.debug(`Tokens usados: ${completion.usage?.total_tokens}`);

    return response;
  } catch (error) {
    this.logger.error('Error generando respuesta:', error);
    throw error;
  }
}

/**
 * Genera respuesta con streaming
 */
async *generateChatResponseStream(
  messages: ChatCompletionMessageParam[],
  options: ChatOptions = {},
): AsyncGenerator<string> {
  try {
    const config = this.openaiConfig.getModelConfig('chat');

    const stream = await this.client.chat.completions.create({
      model: config.model,
      messages: options.systemPrompt
        ? [{ role: 'system', content: options.systemPrompt }, ...messages]
        : messages,
      temperature: options.temperature ?? config.temperature,
      max_tokens: options.maxTokens ?? config.max_tokens,
      stream: true,
    });

    for await (const chunk of stream) {
      const content = chunk.choices[0]?.delta?.content || '';
      if (content) {
        yield content;
      }
    }
  } catch (error) {
    this.logger.error('Error en streaming:', error);
    throw error;
  }
}

/**
 * Genera embeddings para texto
 */
async generateEmbedding(text: string): Promise<number[]> {
  try {
    const config = this.openaiConfig.getModelConfig('embedding');

    const response = await this.client.embeddings.create({
      model: config.model,
      input: text,
      dimensions: config.dimensions,
    });

    return response.data[0].embedding;
  } catch (error) {
    this.logger.error('Error generando embedding:', error);
```

```
    throw error;
  }
}

/**
 * Cuenta tokens de un texto
 */
countTokens(text: string, model: string = 'gpt-4'): number {
  try {
    const encoding = encoding_for_model(model as any);
    const tokens = encoding.encode(text);
    encoding.free();
    return tokens.length;
  } catch (error) {
    // Fallback: aproximación
    return Math.ceil(text.length / 4);
  }
}

/**
 * Extrae información estructurada usando Function Calling
 */
async extractStructuredData<T>(
  text: string,
  schema: any,
  description: string,
): Promise<T> {
  try {
    const completion = await this.client.chat.completions.create({
      model: 'gpt-4-turbo-preview',
      messages: [
        {
          role: 'system',
          content: 'Eres un asistente que extrae información estructurada de texto.',
        },
        {
          role: 'user',
          content: text,
        },
      ],
      functions: [
        {
          name: 'extract_data',
          description,
          parameters: schema,
        },
      ],
      function_call: { name: 'extract_data' },
    });

    const functionCall = completion.choices[0]?.message?.function_call;
    if (functionCall?.arguments) {
      return JSON.parse(functionCall.arguments);
    }

    throw new Error('No se pudo extraer información estructurada');
  } catch (error) {
    this.logger.error('Error extrayendo datos estructurados:', error);
    throw error;
```

```
    }
  }
}
```

🔗 LangChain Implementation

Configuración de LangChain

Archivo: `apps/backend/src/modules/ai/services/langchain.service.ts`
Copiar

```
import { Injectable, Logger } from '@nestjs/common';
import { ChatOpenAI } from '@langchain/openai';
import { ConversationChain } from 'langchain/chains';
import { BufferMemory, ChatMessageHistory } from 'langchain/memory';
import { PromptTemplate } from '@langchain/core/prompts';
import { HumanMessage, AIMessage, SystemMessage } from '@langchain/core/messages';
import { PrismaService } from '@/infrastructure/database/prisma.service';

@Injectable()
export class LangChainService {
  private readonly logger = new Logger(LangChainService.name);
  private readonly llm: ChatOpenAI;

  constructor(private prisma: PrismaService) {
    this.llm = new ChatOpenAI({
      modelName: 'gpt-4-turbo-preview',
      temperature: 0.7,
      maxTokens: 500,
      openAIApiKey: process.env.OPENAI_API_KEY,
    });
  }

  /**
   * Crea una cadena de conversación con memoria
   */
  async createConversationChain(
    conversationId: string,
    systemPrompt: string,
  ): Promise<ConversationChain> {
    // Cargar historial de mensajes desde la base de datos
    const messages = await this.prisma.message.findMany({
      where: { conversationId },
      orderBy: { createdAt: 'asc' },
      take: 20, // Últimos 20 mensajes para contexto
    });

    // Convertir a formato LangChain
    const chatHistory = new ChatMessageHistory(
      messages.map((msg) =>
        msg.fromMe
          ? new AIMessage(msg.content)
          : new HumanMessage(msg.content),
```

```
      ),
    );

    // Crear memoria con el historial
    const memory = new BufferMemory({
      chatHistory,
      returnMessages: true,
      memoryKey: 'history',
    });

    // Crear la cadena
    const chain = new ConversationChain({
      llm: this.llm,
      memory,
      prompt: PromptTemplate.fromTemplate(`
${systemPrompt}

Historial de conversación:
{history}

Usuario: {input}
Asistente:`),
    });

    return chain;
  }

  /**
   * Genera respuesta usando la cadena de conversación
   */
  async generateResponse(
    conversationId: string,
    userMessage: string,
    systemPrompt: string,
  ): Promise<string> {
    try {
      const chain = await this.createConversationChain(
        conversationId,
        systemPrompt,
      );

      const response = await chain.call({
        input: userMessage,
      });

      return response.response;
    } catch (error) {
      this.logger.error('Error generando respuesta con LangChain:', error);
      throw error;
    }
  }

  /**
   * Genera respuesta con streaming
   */
  async *generateResponseStream(
    conversationId: string,
    userMessage: string,
    systemPrompt: string,
```

```
  ): AsyncGenerator<string> {
    try {
      const chain = await this.createConversationChain(
        conversationId,
        systemPrompt,
      );

      const stream = await chain.stream({
        input: userMessage,
      });

      for await (const chunk of stream) {
        yield chunk.response || '';
      }
    } catch (error) {
      this.logger.error('Error en streaming con LangChain:', error);
      throw error;
    }
  }
}
```

Chains Personalizadas

Archivo: `apps/backend/src/modules/ai/chains/customer-support.chain.ts`
Copiar

```
import { ChatOpenAI } from '@langchain/openai';
import { PromptTemplate } from '@langchain/core/prompts';
import { LLMChain } from 'langchain/chains';
import { z } from 'zod';
import { StructuredOutputParser } from 'langchain/output_parsers';

/**
 * Chain para atención al cliente
 */
export class CustomerSupportChain {
  private chain: LLMChain;

  constructor() {
    const llm = new ChatOpenAI({
      modelName: 'gpt-4-turbo-preview',
      temperature: 0.7,
    });

    // Schema de salida estructurada
    const outputSchema = z.object({
      response: z.string().describe('Respuesta al cliente'),
      intent: z.enum([
        'question',
        'complaint',
        'request',
        'feedback',
        'other',
      ]).describe('Intención del mensaje'),
      sentiment: z.enum(['positive', 'neutral', 'negative']).describe('Sentimiento'),
      needsHumanAgent: z.boolean().describe('¿Requiere agente humano?'),
      suggestedActions: z.array(z.string()).describe('Acciones sugeridas'),
```

```
    });

    const parser = StructuredOutputParser.fromZodSchema(outputSchema);

    const prompt = PromptTemplate.fromTemplate(`
Eres un asistente virtual de atención al cliente para OpenTalkWisp, un CRM omnicanal.

Información del cliente:
Nombre: {customerName}
Empresa: {customerCompany}
Historial: {customerHistory}

Mensaje del cliente:
{customerMessage}

Responde de manera profesional, empática y útil. Si el problema es complejo o el cliente
está molesto, sugiere derivar a un agente humano.

{format_instructions}
`);

    this.chain = new LLMChain({
      llm,
      prompt,
      outputParser: parser,
    });
  }

  async process(input: {
    customerName: string;
    customerCompany?: string;
    customerHistory: string;
    customerMessage: string;
  }) {
    const result = await this.chain.call({
      ...input,
      format_instructions: 'Responde en formato JSON con los campos especificados.',
    });

    return result.text;
  }
}
```

🔄 Flujos Conversacionales

Sistema de Flujos Dinámicos

Archivo: `apps/backend/src/modules/ai/services/flow-executor.service.ts`

Copiar

```
import { Injectable, Logger } from '@nestjs/common';
import { PrismaService } from '@/infrastructure/database/prisma.service';
import { LangChainService } from './langchain.service';
```

```
import { FlowNodeType } from '@prisma/client';

interface FlowNode {
  id: string;
  type: FlowNodeType;
  data: any;
  next?: string | string[]; // ID del siguiente nodo o array de IDs
}

interface FlowExecutionContext {
  variables: Record<string, any>;
  currentNodeId: string;
  conversationId: string;
  contactId: string;
}

@Injectable()
export class FlowExecutorService {
  private readonly logger = new Logger(FlowExecutorService.name);

  constructor(
    private prisma: PrismaService,
    private langchain: LangChainService,
  ) {}

  /**
   * Ejecuta un flujo completo
   */
  async executeFlow(
    flowId: string,
    conversationId: string,
    contactId: string,
    initialVariables: Record<string, any> = {},
  ): Promise<void> {
    // Cargar el flujo
    const flow = await this.prisma.flow.findUnique({
      where: { id: flowId },
    });

    if (!flow || !flow.isActive) {
      throw new Error('Flujo no encontrado o inactivo');
    }

    // Crear ejecución
    const execution = await this.prisma.flowExecution.create({
      data: {
        flowId,
        conversationId,
        contactId,
        status: 'running',
        variables: initialVariables,
        startedAt: new Date(),
      },
    });

    const nodes = flow.nodes as FlowNode[];
    const startNode = nodes.find((n) => n.type === 'START');

    if (!startNode) {
```

```
      throw new Error('Flujo sin nodo START');
    }

    const context: FlowExecutionContext = {
      variables: initialVariables,
      currentNodeId: startNode.id,
      conversationId,
      contactId,
    };

    try {
      await this.executeNode(nodes, context, execution.id);

      // Marcar como completado
      await this.prisma.flowExecution.update({
        where: { id: execution.id },
        data: {
          status: 'completed',
          completedAt: new Date(),
        },
      });
    } catch (error) {
      this.logger.error('Error ejecutando flujo:', error);

      await this.prisma.flowExecution.update({
        where: { id: execution.id },
        data: {
          status: 'failed',
          errorMessage: error.message,
          completedAt: new Date(),
        },
      });

      throw error;
    }
  }

  /**
   * Ejecuta un nodo individual
   */
  private async executeNode(
    nodes: FlowNode[],
    context: FlowExecutionContext,
    executionId: string,
  ): Promise<void> {
    const node = nodes.find((n) => n.id === context.currentNodeId);

    if (!node) {
      this.logger.warn(`Nodo ${context.currentNodeId} no encontrado`);
      return;
    }

    this.logger.debug(`Ejecutando nodo: ${node.type} (${node.id})`);

    // Actualizar nodo actual en la ejecución
    await this.prisma.flowExecution.update({
      where: { id: executionId },
      data: { currentNodeId: node.id },
    });
```

```
  switch (node.type) {
    case 'MESSAGE':
      await this.executeMessageNode(node, context);
      break;

    case 'QUESTION':
      await this.executeQuestionNode(node, context);
      return; // Esperar respuesta del usuario

    case 'CONDITION':
      await this.executeConditionNode(node, context);
      break;

    case 'ACTION':
      await this.executeActionNode(node, context);
      break;

    case 'AI_RESPONSE':
      await this.executeAIResponseNode(node, context);
      break;

    case 'WEBHOOK':
      await this.executeWebhookNode(node, context);
      break;

    case 'DELAY':
      await this.executeDelayNode(node, context);
      break;

    case 'END':
      this.logger.debug('Flujo terminado');
      return;

    default:
      this.logger.warn(`Tipo de nodo desconocido: ${node.type}`);
  }

  // Continuar con el siguiente nodo
  if (node.next) {
    const nextNodeId = Array.isArray(node.next) ? node.next[0] : node.next;
    context.currentNodeId = nextNodeId;
    await this.executeNode(nodes, context, executionId);
  }
}

/**
 * Ejecuta nodo de mensaje
 */
private async executeMessageNode(
  node: FlowNode,
  context: FlowExecutionContext,
): Promise<void> {
  const message = this.replaceVariables(node.data.message, context.variables);

  await this.prisma.message.create({
    data: {
      conversationId: context.conversationId,
      contactId: context.contactId,
```

```
      whatsappInstanceId: await this.getWhatsAppInstanceId(context.conversationId),
      type: 'TEXT',
      content: message,
      fromMe: true,
      status: 'SENT',
      provider: 'META_CLOUD',
      aiGenerated: true,
    },
  });

  this.logger.debug(`Mensaje enviado: ${message}`);
}

/**
 * Ejecuta nodo de pregunta
 */
private async executeQuestionNode(
  node: FlowNode,
  context: FlowExecutionContext,
): Promise<void> {
  const question = this.replaceVariables(
    node.data.question,
    context.variables,
  );

  await this.prisma.message.create({
    data: {
      conversationId: context.conversationId,
      contactId: context.contactId,
      whatsappInstanceId: await this.getWhatsAppInstanceId(context.conversationId),
      type: 'TEXT',
      content: question,
      fromMe: true,
      status: 'SENT',
      provider: 'META_CLOUD',
      aiGenerated: true,
    },
  });

  // Aquí se pausa la ejecución esperando respuesta del usuario
  // La respuesta se capturará en el webhook de mensajes entrantes
  this.logger.debug('Esperando respuesta del usuario...');
}

/**
 * Ejecuta nodo de condición
 */
private async executeConditionNode(
  node: FlowNode,
  context: FlowExecutionContext,
): Promise<void> {
  const { condition, trueNext, falseNext } = node.data;

  // Evaluar condición (simple)
  const result = this.evaluateCondition(condition, context.variables);

  // Cambiar el siguiente nodo según el resultado
  if (result) {
    context.currentNodeId = trueNext;
```

```
  } else {
    context.currentNodeId = falseNext;
  }
}

/**
 * Ejecuta nodo de acción
 */
private async executeActionNode(
  node: FlowNode,
  context: FlowExecutionContext,
): Promise<void> {
  const { action, params } = node.data;

  switch (action) {
    case 'update_contact':
      await this.prisma.contact.update({
        where: { id: context.contactId },
        data: params,
      });
      break;

    case 'create_deal':
      await this.prisma.deal.create({
        data: {
          ...params,
          contactId: context.contactId,
        },
      });
      break;

    case 'add_tag':
      const contact = await this.prisma.contact.findUnique({
        where: { id: context.contactId },
      });
      await this.prisma.contact.update({
        where: { id: context.contactId },
        data: {
          tags: [...contact.tags, params.tag],
        },
      });
      break;

    case 'assign_agent':
      await this.prisma.conversation.update({
        where: { id: context.conversationId },
        data: {
          assignedAgentId: params.agentId,
        },
      });
      break;

    default:
      this.logger.warn(`Acción desconocida: ${action}`);
  }
}

/**
 * Ejecuta nodo de respuesta con IA
```

```
 */
private async executeAIResponseNode(
  node: FlowNode,
  context: FlowExecutionContext,
): Promise<void> {
  const { systemPrompt } = node.data;

  // Obtener último mensaje del usuario
  const lastUserMessage = await this.prisma.message.findFirst({
    where: {
      conversationId: context.conversationId,
      fromMe: false,
    },
    orderBy: { createdAt: 'desc' },
  });

  if (!lastUserMessage) {
    this.logger.warn('No hay mensaje del usuario para generar respuesta IA');
    return;
  }

  // Generar respuesta con IA
  const aiResponse = await this.langchain.generateResponse(
    context.conversationId,
    lastUserMessage.content,
    systemPrompt,
  );

  // Enviar respuesta
  await this.prisma.message.create({
    data: {
      conversationId: context.conversationId,
      contactId: context.contactId,
      whatsappInstanceId: await this.getWhatsAppInstanceId(context.conversationId),
      type: 'TEXT',
      content: aiResponse,
      fromMe: true,
      status: 'SENT',
      provider: 'META_CLOUD',
      aiGenerated: true,
    },
  });
}

/**
 * Ejecuta nodo de webhook
 */
private async executeWebhookNode(
  node: FlowNode,
  context: FlowExecutionContext,
): Promise<void> {
  const { url, method, headers, body } = node.data;

  try {
    const response = await fetch(url, {
      method: method || 'POST',
      headers: {
        'Content-Type': 'application/json',
        ...headers,
```

```
      },
      body: JSON.stringify(this.replaceVariables(body, context.variables)),
    });

    const data = await response.json();

    // Guardar respuesta en variables
    context.variables.webhookResponse = data;

    this.logger.debug('Webhook ejecutado exitosamente');
  } catch (error) {
    this.logger.error('Error ejecutando webhook:', error);
    throw error;
  }
}

/**
 * Ejecuta nodo de delay
 */
private async executeDelayNode(
  node: FlowNode,
  context: FlowExecutionContext,
): Promise<void> {
  const { duration } = node.data; // en segundos

  this.logger.debug(`Esperando ${duration} segundos...`);
  await new Promise((resolve) => setTimeout(resolve, duration * 1000));
}

/**
 * Reemplaza variables en un string
 */
private replaceVariables(
  text: string,
  variables: Record<string, any>,
): string {
  return text.replace(/\{\{(\w+)\}\}/g, (match, key) => {
    return variables[key] || match;
  });
}

/**
 * Evalúa una condición simple
 */
private evaluateCondition(
  condition: string,
  variables: Record<string, any>,
): boolean {
  // Implementación simple de evaluación de condiciones
  // Formato: "variable operator value"
  // Ejemplo: "leadScore >= 70"

  const parts = condition.split(' ');
  if (parts.length !== 3) return false;

  const [varName, operator, value] = parts;
  const varValue = variables[varName];

  switch (operator) {
```

```
      case '==':
        return varValue == value;
      case '!=':
        return varValue != value;
      case '>':
        return Number(varValue) > Number(value);
      case '<':
        return Number(varValue) < Number(value);
      case '>=':
        return Number(varValue) >= Number(value);
      case '<=':
        return Number(varValue) <= Number(value);
      case 'contains':
        return String(varValue).includes(value);
      default:
        return false;
    }
  }

  /**
   * Obtiene el ID de la instancia de WhatsApp
   */
  private async getWhatsAppInstanceId(conversationId: string): Promise<string> {
    const conversation = await this.prisma.conversation.findUnique({
      where: { id: conversationId },
      select: { whatsappInstanceId: true },
    });

    return conversation.whatsappInstanceId;
  }
}
```

🎯 Lead Scoring con IA

Archivo: `apps/backend/src/modules/ai/services/lead-scoring.service.ts`
Copiar

```
import { Injectable, Logger } from '@nestjs/common';
import { OpenAIService } from './openai.service';
import { PrismaService } from '@/infrastructure/database/prisma.service';

interface LeadScoringFactors {
  conversationQuality: number; // 0-100
  responseTime: number; // 0-100
  engagementLevel: number; // 0-100
  intentSignals: number; // 0-100
  demographicFit: number; // 0-100
}

@Injectable()
export class LeadScoringService {
  private readonly logger = new Logger(LeadScoringService.name);

  constructor(
```

```
  private openai: OpenAIService,
  private prisma: PrismaService,
) {}

/**
 * Calcula el score de un lead usando IA
 */
async calculateLeadScore(contactId: string): Promise<number> {
  try {
    // Obtener datos del contacto
    const contact = await this.prisma.contact.findUnique({
      where: { id: contactId },
      include: {
        conversations: {
          include: {
            messages: {
              orderBy: { createdAt: 'desc' },
              take: 50,
            },
          },
        },
        deals: true,
      },
    });

    if (!contact) {
      throw new Error('Contacto no encontrado');
    }

    // Calcular factores individuales
    const factors = await this.calculateScoringFactors(contact);

    // Usar IA para análisis cualitativo
    const aiScore = await this.analyzeWithAI(contact, factors);

    // Combinar scores (70% factores, 30% IA)
    const finalScore = Math.round(
      this.calculateWeightedScore(factors) * 0.7 + aiScore * 0.3,
    );

    // Actualizar en base de datos
    await this.prisma.contact.update({
      where: { id: contactId },
      data: { leadScore: finalScore },
    });

    this.logger.debug(`Lead score calculado para ${contact.name}: ${finalScore}`);

    return finalScore;
  } catch (error) {
    this.logger.error('Error calculando lead score:', error);
    throw error;
  }
}

/**
 * Calcula factores de scoring
 */
private async calculateScoringFactors(
```

```
  contact: any,
): Promise<LeadScoringFactors> {
  // 1. Calidad de conversación
  const conversationQuality = this.calculateConversationQuality(
    contact.conversations,
  );

  // 2. Tiempo de respuesta
  const responseTime = this.calculateResponseTimeScore(contact.conversations);

  // 3. Nivel de engagement
  const engagementLevel = this.calculateEngagementLevel(contact);

  // 4. Señales de intención
  const intentSignals = this.calculateIntentSignals(contact.conversations);

  // 5. Fit demográfico
  const demographicFit = this.calculateDemographicFit(contact);

  return {
    conversationQuality,
    responseTime,
    engagementLevel,
    intentSignals,
    demographicFit,
  };
}

/**
 * Calcula calidad de conversación
 */
private calculateConversationQuality(conversations: any[]): number {
  if (conversations.length === 0) return 0;

  let totalScore = 0;

  for (const conv of conversations) {
    const messages = conv.messages || [];
    const userMessages = messages.filter((m) => !m.fromMe);

    // Factores:
    // - Longitud de mensajes (mensajes largos = más interés)
    // - Cantidad de mensajes
    // - Uso de palabras clave positivas

    const avgMessageLength =
      userMessages.reduce((sum, m) => sum + m.content.length, 0) /
      (userMessages.length || 1);

    const lengthScore = Math.min((avgMessageLength / 100) * 100, 100);
    const quantityScore = Math.min((userMessages.length / 10) * 100, 100);

    totalScore += (lengthScore + quantityScore) / 2;
  }

  return Math.round(totalScore / conversations.length);
}

/**
```

```
 * Calcula score de tiempo de respuesta
 */
private calculateResponseTimeScore(conversations: any[]): number {
  if (conversations.length === 0) return 0;

  let totalScore = 0;

  for (const conv of conversations) {
    const messages = conv.messages || [];

    for (let i = 1; i < messages.length; i++) {
      const prev = messages[i - 1];
      const curr = messages[i];

      // Si es respuesta del usuario a nuestro mensaje
      if (prev.fromMe && !curr.fromMe) {
        const responseTime =
          new Date(curr.createdAt).getTime() -
          new Date(prev.createdAt).getTime();

        // Respuestas rápidas = más interés
        // < 5 min = 100, < 30 min = 80, < 1 hora = 60, < 1 día = 40, > 1 día = 20
        const minutes = responseTime / 60000;

        let score = 0;
        if (minutes < 5) score = 100;
        else if (minutes < 30) score = 80;
        else if (minutes < 60) score = 60;
        else if (minutes < 1440) score = 40;
        else score = 20;

        totalScore += score;
      }
    }
  }

  return Math.round(totalScore / (conversations.length || 1));
}

/**
 * Calcula nivel de engagement
 */
private calculateEngagementLevel(contact: any): number {
  let score = 0;

  // Tiene email: +10
  if (contact.email) score += 10;

  // Tiene empresa: +10
  if (contact.company) score += 10;

  // Tiene tags: +5 por tag (max 20)
  score += Math.min(contact.tags.length * 5, 20);

  // Cantidad de conversaciones: +5 por conversación (max 30)
  score += Math.min(contact.conversations.length * 5, 30);

  // Tiene deals: +30
  if (contact.deals && contact.deals.length > 0) score += 30;
```

```
  return Math.min(score, 100);
}

/**
 * Calcula señales de intención
 */
private calculateIntentSignals(conversations: any[]): number {
  const intentKeywords = {
    high: ['comprar', 'precio', 'cotización', 'contratar', 'cuánto cuesta', 'plan'],
    medium: ['información', 'detalles', 'características', 'demo', 'prueba'],
    low: ['hola', 'ayuda', 'consulta', 'pregunta'],
  };

  let score = 0;
  let totalMessages = 0;

  for (const conv of conversations) {
    const messages = conv.messages || [];
    const userMessages = messages.filter((m) => !m.fromMe);

    for (const msg of userMessages) {
      totalMessages++;
      const content = msg.content.toLowerCase();

      if (intentKeywords.high.some((kw) => content.includes(kw))) {
        score += 10;
      } else if (intentKeywords.medium.some((kw) => content.includes(kw))) {
        score += 5;
      } else if (intentKeywords.low.some((kw) => content.includes(kw))) {
        score += 2;
      }
    }
  }

  return Math.min(Math.round((score / (totalMessages || 1)) * 10), 100);
}

/**
 * Calcula fit demográfico
 */
private calculateDemographicFit(contact: any): number {
  let score = 50; // Base

  // Tiene empresa: +20
  if (contact.company) score += 20;

  // Tiene puesto de trabajo: +15
  if (contact.jobTitle) score += 15;

  // Tiene ubicación: +15
  if (contact.city && contact.country) score += 15;

  return Math.min(score, 100);
}

/**
 * Analiza con IA para score cualitativo
 */
```

```
  private async analyzeWithAI(
    contact: any,
    factors: LeadScoringFactors,
  ): Promise<number> {
    try {
      // Preparar contexto para la IA
      const recentMessages = contact.conversations
        .flatMap((c) => c.messages)
        .filter((m) => !m.fromMe)
        .slice(0, 10)
        .map((m) => m.content)
        .join('\n');

      const prompt = `
Analiza la calidad de este lead basándote en sus mensajes recientes y factores
cuantitativos.

Información del contacto:
- Nombre: ${contact.name || 'Desconocido'}
- Empresa: ${contact.company || 'No especificada'}
- Puesto: ${contact.jobTitle || 'No especificado'}

Mensajes recientes del contacto:
${recentMessages || 'Sin mensajes'}

Factores cuantitativos (0-100):
- Calidad de conversación: ${factors.conversationQuality}
- Tiempo de respuesta: ${factors.responseTime}
- Nivel de engagement: ${factors.engagementLevel}
- Señales de intención: ${factors.intentSignals}
- Fit demográfico: ${factors.demographicFit}

Basándote en toda esta información, proporciona un score de 0 a 100 que represente la
probabilidad de que este lead se convierta en cliente.

Considera:
- Tono y profesionalismo de los mensajes
- Claridad en las necesidades expresadas
- Nivel de interés demostrado
- Capacidad de compra implícita

Responde SOLO con el número del score (0-100).
`;

      const response = await this.openai.generateChatResponse(
        [{ role: 'user', content: prompt }],
        { temperature: 0.3, maxTokens: 10 },
      );

      const score = parseInt(response.trim());

      if (isNaN(score) || score < 0 || score > 100) {
        this.logger.warn('IA devolvió score inválido, usando factores solo');
        return this.calculateWeightedScore(factors);
      }

      return score;
    } catch (error) {
      this.logger.error('Error en análisis con IA:', error);
```

```
      return this.calculateWeightedScore(factors);
    }
  }

  /**
   * Calcula score ponderado de factores
   */
  private calculateWeightedScore(factors: LeadScoringFactors): number {
    const weights = {
      conversationQuality: 0.25,
      responseTime: 0.15,
      engagementLevel: 0.20,
      intentSignals: 0.30,
      demographicFit: 0.10,
    };

    const score =
      factors.conversationQuality * weights.conversationQuality +
      factors.responseTime * weights.responseTime +
      factors.engagementLevel * weights.engagementLevel +
      factors.intentSignals * weights.intentSignals +
      factors.demographicFit * weights.demographicFit;

    return Math.round(score);
  }
}
```

☺ Análisis de Sentimiento

Archivo: `apps/backend/src/modules/ai/services/sentiment-analysis.service.ts`
Copiar

```
import { Injectable, Logger } from '@nestjs/common';
import { OpenAIService } from './openai.service';

export interface SentimentResult {
  sentiment: 'positive' | 'neutral' | 'negative';
  score: number; // -1.0 a 1.0
  confidence: number; // 0.0 a 1.0
  emotions: {
    joy?: number;
    anger?: number;
    sadness?: number;
    fear?: number;
    surprise?: number;
  };
}

@Injectable()
export class SentimentAnalysisService {
  private readonly logger = new Logger(SentimentAnalysisService.name);

  constructor(private openai: OpenAIService) {}
```

```
/**
 * Analiza el sentimiento de un mensaje
 */
async analyzeSentiment(text: string): Promise<SentimentResult> {
  try {
    const schema = {
      type: 'object',
      properties: {
        sentiment: {
          type: 'string',
          enum: ['positive', 'neutral', 'negative'],
          description: 'Sentimiento general del mensaje',
        },
        score: {
          type: 'number',
          description: 'Score de -1.0 (muy negativo) a 1.0 (muy positivo)',
        },
        confidence: {
          type: 'number',
          description: 'Confianza en el análisis (0.0 a 1.0)',
        },
        emotions: {
          type: 'object',
          properties: {
            joy: { type: 'number' },
            anger: { type: 'number' },
            sadness: { type: 'number' },
            fear: { type: 'number' },
            surprise: { type: 'number' },
          },
          description: 'Emociones detectadas (0.0 a 1.0)',
        },
      },
      required: ['sentiment', 'score', 'confidence', 'emotions'],
    };

    const result = await this.openai.extractStructuredData<SentimentResult>(
      text,
      schema,
      'Analiza el sentimiento y emociones del siguiente mensaje',
    );

    return result;
  } catch (error) {
    this.logger.error('Error analizando sentimiento:', error);

    // Fallback simple
    return {
      sentiment: 'neutral',
      score: 0,
      confidence: 0.5,
      emotions: {},
    };
  }
}

/**
 * Analiza el sentimiento de una conversación completa
 */
```

```
  async analyzeConversationSentiment(
    messages: Array<{ content: string; fromMe: boolean }>,
  ): Promise<SentimentResult> {
    // Analizar solo mensajes del usuario
    const userMessages = messages
      .filter((m) => !m.fromMe)
      .map((m) => m.content)
      .join('\n');

    if (!userMessages) {
      return {
        sentiment: 'neutral',
        score: 0,
        confidence: 0,
        emotions: {},
      };
    }

    return this.analyzeSentiment(userMessages);
  }
}
```

🔍 RAG (Retrieval Augmented Generation)

Archivo: `apps/backend/src/modules/ai/services/rag.service.ts`
Copiar

```
import { Injectable, Logger } from '@nestjs/common';
import { OpenAIService } from './openai.service';
import { PineconeClient } from '@pinecone-database/pinecone';
import { Document } from 'langchain/document';
import { RecursiveCharacterTextSplitter } from 'langchain/text_splitter';

@Injectable()
export class RAGService {
  private readonly logger = new Logger(RAGService.name);
  private pinecone: PineconeClient;
  private indexName = 'opentalkwisp-knowledge';

  constructor(private openai: OpenAIService) {
    this.initializePinecone();
  }

  /**
   * Inicializa Pinecone
   */
  private async initializePinecone() {
    this.pinecone = new PineconeClient();
    await this.pinecone.init({
      apiKey: process.env.PINECONE_API_KEY,
      environment: process.env.PINECONE_ENVIRONMENT,
    });
  }
```

```
/**
 * Indexa documentos en la base de conocimiento
 */
async indexDocuments(documents: Array<{ content: string; metadata: any }>) {
  try {
    const index = this.pinecone.Index(this.indexName);

    // Dividir documentos en chunks
    const textSplitter = new RecursiveCharacterTextSplitter({
      chunkSize: 1000,
      chunkOverlap: 200,
    });

    const chunks: Document[] = [];

    for (const doc of documents) {
      const splits = await textSplitter.createDocuments(
        [doc.content],
        [doc.metadata],
      );
      chunks.push(...splits);
    }

    // Generar embeddings
    const vectors = [];

    for (let i = 0; i < chunks.length; i++) {
      const chunk = chunks[i];
      const embedding = await this.openai.generateEmbedding(chunk.pageContent);

      vectors.push({
        id: `doc-${Date.now()}-${i}`,
        values: embedding,
        metadata: {
          text: chunk.pageContent,
          ...chunk.metadata,
        },
      });
    }

    // Subir a Pinecone
    await index.upsert({ upsertRequest: { vectors } });

    this.logger.log(`${vectors.length} chunks indexados exitosamente`);
  } catch (error) {
    this.logger.error('Error indexando documentos:', error);
    throw error;
  }
}

/**
 * Busca documentos relevantes
 */
async searchRelevantDocuments(
  query: string,
  topK: number = 5,
): Promise<Array<{ text: string; score: number; metadata: any }>> {
  try {
    const index = this.pinecone.Index(this.indexName);
```

```
      // Generar embedding de la query
      const queryEmbedding = await this.openai.generateEmbedding(query);

      // Buscar en Pinecone
      const results = await index.query({
        queryRequest: {
          vector: queryEmbedding,
          topK,
          includeMetadata: true,
        },
      });

      return results.matches.map((match) => ({
        text: match.metadata.text as string,
        score: match.score,
        metadata: match.metadata,
      }));
    } catch (error) {
      this.logger.error('Error buscando documentos:', error);
      return [];
    }
  }

  /**
   * Genera respuesta usando RAG
   */
  async generateResponseWithRAG(
    question: string,
    systemPrompt?: string,
  ): Promise<string> {
    try {
      // Buscar documentos relevantes
      const relevantDocs = await this.searchRelevantDocuments(question);

      if (relevantDocs.length === 0) {
        return await this.openai.generateChatResponse(
          [{ role: 'user', content: question }],
          { systemPrompt },
        );
      }

      // Construir contexto
      const context = relevantDocs
        .map((doc, i) => `[${i + 1}] ${doc.text}`)
        .join('\n\n');

      // Generar respuesta con contexto
      const prompt = `
Contexto de la base de conocimiento:
${context}

Pregunta del usuario:
${question}

Responde basándote en el contexto proporcionado. Si la información no está en el contexto,
indícalo claramente.
`;
```

```
      return await this.openai.generateChatResponse(
        [{ role: 'user', content: prompt }],
        { systemPrompt },
      );
    } catch (error) {
      this.logger.error('Error generando respuesta con RAG:', error);
      throw error;
    }
  }
}
```

📝 Prompts del Sistema

Archivo: `apps/backend/src/modules/ai/prompts/system-prompts.ts`

Copiar

```
export const SYSTEM_PROMPTS = {
  /**
   * Prompt para asistente de ventas
   */
  SALES_ASSISTANT: `
Eres un asistente virtual de ventas profesional y amigable para OpenTalkWisp.

Tu objetivo es:
1. Calificar leads haciendo preguntas relevantes
2. Identificar necesidades del cliente
3. Presentar soluciones apropiadas
4. Guiar hacia el cierre de venta

Directrices:
- Sé empático y escucha activamente
- Haz preguntas abiertas para entender necesidades
- No seas agresivo, construye confianza
- Si el cliente no está listo, ofrece contenido educativo
- Deriva a un agente humano cuando sea necesario

Información del producto:
- OpenTalkWisp es un CRM omnicanal con IA
- Integra WhatsApp, Facebook e Instagram
- Automatiza respuestas y califica leads
- Planes desde $299 MXN/mes

Responde siempre en español de manera profesional pero cercana.
`,

  /**
   * Prompt para soporte técnico
   */
  TECHNICAL_SUPPORT: `
Eres un asistente de soporte técnico experto en OpenTalkWisp.

Tu objetivo es:
1. Diagnosticar problemas técnicos
2. Proporcionar soluciones paso a paso
```

```
3. Escalar a soporte humano cuando sea necesario

Directrices:
- Sé paciente y claro en tus explicaciones
- Usa lenguaje simple, evita jerga técnica
- Proporciona pasos numerados
- Verifica que el problema se resolvió
- Si no puedes resolver, deriva a un agente

Problemas comunes:
- Conexión de WhatsApp
- Envío de mensajes
- Sincronización de contactos
- Configuración de campañas

Responde siempre en español de manera clara y estructurada.
`,

  /**
   * Prompt para análisis de intención
   */
  INTENT_ANALYSIS: `
Analiza la intención del mensaje del usuario y clasifícala en una de estas categorías:

1. SALES_INQUIRY - Consulta sobre productos/servicios
2. SUPPORT_REQUEST - Solicitud de ayuda técnica
3. COMPLAINT - Queja o insatisfacción
4. FEEDBACK - Retroalimentación o sugerencia
5. GENERAL_QUESTION - Pregunta general
6. GREETING - Saludo inicial
7. FAREWELL - Despedida
8. OTHER - Otro tipo de mensaje

Responde SOLO con la categoría en mayúsculas.
`,

  /**
   * Prompt para extracción de entidades
   */
  ENTITY_EXTRACTION: `
Extrae las siguientes entidades del mensaje del usuario:

- name: Nombre de la persona
- company: Nombre de la empresa
- email: Dirección de email
- phone: Número de teléfono
- location: Ciudad o país
- product_interest: Producto o servicio de interés
- budget: Presupuesto mencionado
- timeline: Plazo o fecha mencionada

Responde en formato JSON con las entidades encontradas.
Si una entidad no está presente, omítela del JSON.
`,

  /**
   * Prompt para resumen de conversación
   */
  CONVERSATION_SUMMARY: `
```

```
Resume la siguiente conversación de manera concisa.

Incluye:
1. Tema principal de la conversación
2. Necesidades o problemas expresados
3. Soluciones propuestas
4. Próximos pasos acordados
5. Estado del lead (frío/tibio/caliente)

Mantén el resumen en máximo 3 párrafos.
`,

  /**
   * Prompt para generación de respuestas contextuales
   */
  CONTEXTUAL_RESPONSE: `
Genera una respuesta apropiada basándote en:

1. El mensaje actual del usuario
2. El historial de la conversación
3. El perfil del contacto
4. El objetivo de la conversación (venta/soporte/información)

La respuesta debe:
- Ser natural y conversacional
- Mantener el contexto de mensajes anteriores
- Avanzar hacia el objetivo
- Incluir una pregunta o call-to-action cuando sea apropiado
- Ser concisa (máximo 3 párrafos)

Responde en español con un tono profesional pero amigable.
`,
};
```

💰 Optimización y Costos

Estrategias de Optimización

Archivo: `apps/backend/src/modules/ai/services/cost-optimizer.service.ts`

Copiar

```
import { Injectable, Logger } from '@nestjs/common';
import { OpenAIService } from './openai.service';

@Injectable()
export class CostOptimizerService {
  private readonly logger = new Logger(CostOptimizerService.name);

  constructor(private openai: OpenAIService) {}

  /**
   * Optimiza el contexto para reducir tokens
   */
```

```
  optimizeContext(messages: string[], maxTokens: number = 2000): string[] {
    const optimized: string[] = [];
    let totalTokens = 0;

    // Priorizar mensajes más recientes
    for (let i = messages.length - 1; i >= 0; i--) {
      const msg = messages[i];
      const tokens = this.openai.countTokens(msg);

      if (totalTokens + tokens <= maxTokens) {
        optimized.unshift(msg);
        totalTokens += tokens;
      } else {
        break;
      }
    }

    this.logger.debug(`Contexto optimizado: ${optimized.length} mensajes, ${totalTokens}
tokens`);

    return optimized;
  }

  /**
   * Decide si usar IA o respuesta predefinida
   */
  shouldUseAI(message: string): boolean {
    // Respuestas simples no requieren IA
    const simplePatterns = [
      /^hola$/i,
      /^adiós$/i,
      /^gracias$/i,
      /^ok$/i,
      /^sí$/i,
      /^no$/i,
    ];

    return !simplePatterns.some((pattern) => pattern.test(message.trim()));
  }

  /**
   * Selecciona el modelo apropiado según la tarea
   */
  selectModel(task: 'simple' | 'complex' | 'analysis'): string {
    const models = {
      simple: 'gpt-3.5-turbo', // Más barato para tareas simples
      complex: 'gpt-4-turbo-preview', // Para tareas complejas
      analysis: 'gpt-4-turbo-preview', // Para análisis precisos
    };

    return models[task];
  }

  /**
   * Calcula el costo estimado de una operación
   */
  estimateCost(inputTokens: number, outputTokens: number, model: string): number {
    // Precios aproximados (actualizar según OpenAI)
    const prices = {
```

```
      'gpt-3.5-turbo': {
        input: 0.0005 / 1000, // $0.0005 por 1K tokens
        output: 0.0015 / 1000,
      },
      'gpt-4-turbo-preview': {
        input: 0.01 / 1000, // $0.01 por 1K tokens
        output: 0.03 / 1000,
      },
    };

    const modelPrices = prices[model] || prices['gpt-4-turbo-preview'];

    const cost =
      inputTokens * modelPrices.input + outputTokens * modelPrices.output;

    return cost;
  }
}
```

Monitoreo de Uso

Archivo: `apps/backend/src/modules/ai/services/usage-monitor.service.ts`

Copiar

```
import { Injectable, Logger } from '@nestjs/common';
import { PrismaService } from '@/infrastructure/database/prisma.service';

interface UsageStats {
  totalRequests: number;
  totalTokens: number;
  totalCost: number;
  byModel: Record<string, { requests: number; tokens: number; cost: number }>;
}

@Injectable()
export class UsageMonitorService {
  private readonly logger = new Logger(UsageMonitorService.name);

  constructor(private prisma: PrismaService) {}

  /**
   * Registra el uso de IA
   */
  async logUsage(data: {
    organizationId: string;
    model: string;
    inputTokens: number;
    outputTokens: number;
    cost: number;
    operation: string;
  }) {
    // Aquí podrías crear una tabla AIUsage en Prisma
    // Por ahora, solo logging
    this.logger.log(
      `AI Usage - Org: ${data.organizationId}, Model: ${data.model}, ` +
      `Tokens: ${data.inputTokens + data.outputTokens}, Cost: $${data.cost.toFixed(4)}`,
    );
  }
```

```
/**
 * Obtiene estadísticas de uso
 */
async getUsageStats(
  organizationId: string,
  startDate: Date,
  endDate: Date,
): Promise<UsageStats> {
  // Implementar consulta a tabla AIUsage
  // Por ahora, retornar mock
  return {
    totalRequests: 1250,
    totalTokens: 125000,
    totalCost: 3.75,
    byModel: {
      'gpt-3.
```